

```
# =====
# PARKFNB - COLAB + GOOGLE DRIVE SETUP
# =====

from google.colab import drive

drive.mount("/content/drive")

print("✅ Google Drive mounted")
```

Mounted at /content/drive
 ✅ Google Drive mounted

```
# =====
# PARKFNB - CELL 2
# CREATE PERMANENT DATASET STRUCTURE
# =====

from pathlib import Path
import json

# Google Drive location
DRIVE_ROOT = Path("/content/drive/MyDrive/ParkFNB")

DATASET_ROOT = DRIVE_ROOT / "ParkFNB_Dataset"

# Create complete YOLO structure
for split in ["train", "val", "test"]:
    (DATASET_ROOT / "images" / split).mkdir(
        parents=True,
        exist_ok=True
    )

    (DATASET_ROOT / "labels" / split).mkdir(
        parents=True,
        exist_ok=True
    )

print("=====")
print("PARKFNB DATASET STRUCTURE READY")
print("=====")

print(DATASET_ROOT)

print("\nFolders:")

for split in ["train", "val", "test"]:
    print(f"  images/{split}")
    print(f"  labels/{split}")

print("\nClasses:")
print("  0 = vehicle")
print("  1 = license_plate")
```

```
=====
PARKFNB DATASET STRUCTURE READY
=====
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset

Folders:
  images/train
  labels/train
  images/val
  labels/val
  images/test
  labels/test

Classes:
  0 = vehicle
  1 = license_plate
```

```
# =====
# PARKFNB - FINAL DATA COLLECTION + YOLO PREPARATION
# =====
# NO TRAINING
# NO INFERENCE
# NO GPU REQUIRED
#
# Classes:
#   0 = vehicle
#   1 = license_plate
```

```

#
# Workflow:
# BMD-45 vehicles
#   +
#   Indian/Kedarsai license plates
#   ↓
# source pools
#   ↓
# synthesis pool
#   ↓
# later: synthetic image generation
#   ↓
# later: final train/val/test
# =====

# =====
# 1. INSTALL DEPENDENCIES
# =====

!pip -q install datasets huggingface_hub kaggle lxml pillow tqdm pyyaml

# =====
# 2. MOUNT GOOGLE DRIVE
# =====

from google.colab import drive
drive.mount('/content/drive')

# =====
# 3. IMPORTS
# =====

import os
import shutil
import zipfile
import tarfile
import json
import random
import hashlib
import subprocess
from pathlib import Path
from PIL import Image
from lxml import etree
from tqdm.auto import tqdm

from datasets import load_dataset

# =====
# 4. CONFIG
# =====

DRIVE_ROOT = Path(
    "/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset"
)

WORK_ROOT = Path("/content/parkfnb_work")

# How many BMD vehicle images to collect.
# 3817 is the amount we previously successfully processed.
BMD_LIMIT = 3817

# Random seed
SEED = 42
random.seed(SEED)

# =====
# 5. DELETE OLD DATASET COMPLETELY
# =====

print("\n=====")
print("CLEANING OLD PARKFNB DATASET")
print("=====")

if DRIVE_ROOT.exists():
    print(f"Deleting:\n{DRIVE_ROOT}")
    shutil.rmtree(DRIVE_ROOT)

if WORK_ROOT.exists():

```

```

    print(f"Deleting temporary work directory:\n{WORK_ROOT}")
    shutil.rmtree(WORK_ROOT)

DRIVE_ROOT.mkdir(parents=True, exist_ok=True)
WORK_ROOT.mkdir(parents=True, exist_ok=True)

print("\nOLD DATASET DELETED.")
print("Fresh workspace created.")

# =====
# 6. CREATE FINAL FOLDER STRUCTURE
# =====

VEHICLE_ROOT = DRIVE_ROOT / "vehicle_base"
PLATE_ROOT   = DRIVE_ROOT / "plate_assets"
MERGED_ROOT  = DRIVE_ROOT / "merged_source"

for root in [VEHICLE_ROOT, PLATE_ROOT, MERGED_ROOT]:

    (root / "images").mkdir(parents=True, exist_ok=True)
    (root / "labels").mkdir(parents=True, exist_ok=True)

# =====
# 7. KAGGLE DATASET DOWNLOAD
# =====

KAGGLE_DIR = WORK_ROOT / "kaggle"
KAGGLE_DIR.mkdir(parents=True, exist_ok=True)

INDIAN_VEHICLE_URL = (
    "https://www.kaggle.com/api/v1/datasets/download/"
    "saisirishan/indian-vehicle-dataset"
)

KEDARSAI_URL = (
    "https://www.kaggle.com/api/v1/datasets/download/"
    "kedarsai/indian-license-plates-with-labels"
)

vehicle_zip = KAGGLE_DIR / "indian_vehicle_dataset.zip"
plate_zip   = KAGGLE_DIR / "kedarsai_plates.zip"

def download_file(url, output):
    if output.exists():
        print(f"Already downloaded: {output}")
        return

    print(f"\nDownloading: \n{url}")

    subprocess.run(
        [
            "wget",
            "-q",
            "--show-progress",
            "-O",
            str(output),
            url
        ],
        check=True
    )

    print(f"Saved: {output}")

print("\n=====")
print("DOWNLOADING INDIAN VEHICLE DATASET")
print("=====")

download_file(
    INDIAN_VEHICLE_URL,
    vehicle_zip
)

print("\n=====")
print("DOWNLOADING KEDARSAI PLATE DATASET")
print("=====")

download_file(

```

```

        KEDARSAI_URL,
        plate_zip
    )

# =====
# 8. EXTRACT KAGGLE DATASETS
# =====

RAW_VEHICLE = WORK_ROOT / "indian_vehicle_raw"
RAW_PLATE    = WORK_ROOT / "kedarsai_raw"

RAW_VEHICLE.mkdir(parents=True, exist_ok=True)
RAW_PLATE.mkdir(parents=True, exist_ok=True)

print("\nExtracting Indian Vehicle Dataset...")

with zipfile.ZipFile(vehicle_zip, "r") as z:
    z.extractall(RAW_VEHICLE)

print("Extracting Kedarsai Dataset...")

with zipfile.ZipFile(plate_zip, "r") as z:
    z.extractall(RAW_PLATE)

print("\nKaggle datasets extracted.")

# =====
# 9. HELPER FUNCTIONS
# =====

IMAGE_EXTENSIONS = {
    ".jpg",
    ".jpeg",
    ".png",
    ".webp",
    ".bmp"
}

def find_images(root):
    """
    Recursively find all image files.
    """
    root = Path(root)

    files = []

    for p in root.rglob("*"):
        if p.is_file() and p.suffix.lower() in IMAGE_EXTENSIONS:
            files.append(p)

    return files

def find_xml_files(root):
    """
    Recursively find Pascal VOC XML files.
    """
    return list(Path(root).rglob("*.xml"))

def clamp(v, low=0.0, high=1.0):
    return max(low, min(high, v))

def bbox_to_yolo(xmin, ymin, xmax, ymax, width, height):
    """
    Pascal VOC pixel bbox → YOLO normalized bbox
    """

    xmin = max(0, min(xmin, width))
    xmax = max(0, min(xmax, width))
    ymin = max(0, min(ymin, height))
    ymax = max(0, min(ymax, height))

    if xmax <= xmin or ymax <= ymin:
        return None

```

```

xc = ((xmin + xmax) / 2.0) / width
yc = ((ymin + ymax) / 2.0) / height

bw = (xmax - xmin) / width
bh = (ymax - ymin) / height

return (
    clamp(xc),
    clamp(yc),
    clamp(bw),
    clamp(bh)
)

def safe_name(name):
    """
    Make filesystem-safe filename.
    """
    return "".join(
        c if c.isalnum() or c in "-_"
        else "_"
        for c in name
    )

def unique_filename(prefix, source_path):
    """
    Generate unique filename so datasets never overwrite each other.
    """
    h = hashlib.md5(
        str(source_path).encode()
    ).hexdigest()[:10]

    return f"{prefix}_{safe_name(source_path.stem)}_{h}"

# =====
# 10. PROCESS INDIAN VEHICLE / PLATE XML DATA
# =====

print("\n=====")
print("PROCESSING INDIAN VEHICLE DATASET")
print("=====")

xml_files = find_xml_files(RAW_VEHICLE)

print(f"XML files found: {len(xml_files)}")

vehicle_count = 0
vehicle_label_count = 0

for xml_path in tqdm(
    xml_files,
    desc="Indian Vehicle XML"
):
    try:
        tree = etree.parse(str(xml_path))
        root = tree.getroot()

        filename_node = root.find("filename")

        if filename_node is None:
            continue

        original_name = filename_node.text

        # Search for corresponding image
        candidates = list(
            RAW_VEHICLE.rglob(original_name)
        )

        if not candidates:

            # Sometimes XML filename differs in extension
            stem = Path(original_name).stem

            candidates = [
                p for p in RAW_VEHICLE.rglob("*")
                if p.is_file()
            ]

```

```

        and p.stem == stem
        and p.suffix.lower() in IMAGE_EXTENSIONS
    ]

if not candidates:
    continue

image_path = candidates[0]

with Image.open(image_path) as im:
    width, height = im.size

yolo_lines = []

for obj in root.findall("object"):

    # IMPORTANT:
    # The original dataset's XML object is the license plate.
    # Therefore:
    #
    # class 1 = license_plate

    bbox = obj.find("bndbox")

    if bbox is None:
        continue

    xmin = float(bbox.findtext("xmin", "0"))
    ymin = float(bbox.findtext("ymin", "0"))
    xmax = float(bbox.findtext("xmax", "0"))
    ymax = float(bbox.findtext("ymax", "0"))

    yolo_bbox = bbox_to_yolo(
        xmin,
        ymin,
        xmax,
        ymax,
        width,
        height
    )

    if yolo_bbox is None:
        continue

    xc, yc, bw, bh = yolo_bbox

    yolo_lines.append(
        f"1 {xc:.6f} {yc:.6f} {bw:.6f} {bh:.6f}"
    )

if not yolo_lines:
    continue

new_base = unique_filename(
    "indian",
    image_path
)

destination_image = (
    PLATE_ROOT / "images" / f"{new_base}{image_path.suffix.lower()}"
)

destination_label = (
    PLATE_ROOT / "labels" / f"{new_base}.txt"
)

shutil.copy2(
    image_path,
    destination_image
)

destination_label.write_text(
    "\n".join(yolo_lines),
    encoding="utf-8"
)

vehicle_count += 1
vehicle_label_count += len(yolo_lines)

except Exception as e:

    print(

```

```

        f"\nSkipped {xml_path.name}: {e}"
    )

print("\nIndian Vehicle Dataset processing complete.")
print(f"Images with plate labels: {vehicle_count}")
print(f"License plate boxes: {vehicle_label_count}")

# =====
# 11. PROCESS KEDARSAI
# =====

print("\n=====")
print("PROCESSING KEDARSAI PLATE DATASET")
print("=====")

# Find image and label directories automatically
kedarsai_images = []
kedarsai_labels = []

for p in RAW_PLATE.rglob("*"):

    if not p.is_file():
        continue

    if p.suffix.lower() in IMAGE_EXTENSIONS:
        kedarsai_images.append(p)

    elif p.suffix.lower() == ".txt":
        kedarsai_labels.append(p)

label_map = {
    p.stem: p
    for p in kedarsai_labels
}

print(f"Images found: {len(kedarsai_images)}")
print(f"TXT labels found: {len(kedarsai_labels)}")

kedarsai_count = 0
kedarsai_boxes = 0

for image_path in tqdm(
    kedarsai_images,
    desc="Kedarsai plates"
):

    label_path = label_map.get(
        image_path.stem
    )

    if label_path is None:
        continue

    try:

        with Image.open(image_path) as im:
            width, height = im.size

        output_lines = []

        raw_lines = label_path.read_text(
            encoding="utf-8",
            errors="ignore"
        ).splitlines()

        for line in raw_lines:

            parts = line.strip().split()

            if len(parts) != 5:
                continue

            try:

                # Existing dataset is expected to be YOLO:

```

```

        # class x_center y_center width height
        old_class = int(float(parts[0]))

        xc = float(parts[1])
        yc = float(parts[2])
        bw = float(parts[3])
        bh = float(parts[4])

    except:
        continue

    # Regardless of original plate class:
    # force everything to our class 1.
    xc = clamp(xc)
    yc = clamp(yc)
    bw = clamp(bw)
    bh = clamp(bh)

    if bw <= 0 or bh <= 0:
        continue

    output_lines.append(
        f"1 {xc:.6f} {yc:.6f} {bw:.6f} {bh:.6f}"
    )

    if not output_lines:
        continue

    new_base = unique_filename(
        "kedar",
        image_path
    )

    destination_image = (
        PLATE_ROOT
        / "images"
        / f"{new_base}{image_path.suffix.lower()}"
    )

    destination_label = (
        PLATE_ROOT
        / "labels"
        / f"{new_base}.txt"
    )

    shutil.copy2(
        image_path,
        destination_image
    )

    destination_label.write_text(
        "\n".join(output_lines),
        encoding="utf-8"
    )

    kedarsai_count += 1
    kedarsai_boxes += len(output_lines)

except Exception as e:

    print(
        f"\nSkipped {image_path.name}: {e}"
    )

print("\nKedarsai processing complete.")
print(f"Images with plate labels: {kedarsai_count}")
print(f"License plate boxes: {kedarsai_boxes}")

# =====
# 12. DOWNLOAD / STREAM BMD-45
# =====

print("\n=====")
print("STREAMING BMD-45")
print("=====")

print(
    "BMD-45 will NOT be fully downloaded."
)
print(

```

```

        f"Only {BMD_LIMIT} vehicle images will be collected."
    )

# BMD-45 has official HuggingFace ImageFolder annotations.
# We use streaming so images are processed one by one.

bmd_stream = load_dataset(
    "iisc-aim/BMD-45",
    split="train",
    streaming=True
)

# =====
# 13. BMD CATEGORY → GENERIC VEHICLE
# =====

# Every BMD-45 object is converted to:
#
# class 0 = vehicle
#
# This is intentional because ParkFNB needs a generic
# vehicle detector before the plate/OCR stage.

BMD_VEHICLE_CLASS = 0

bmd_images = 0
bmd_boxes = 0

# =====
# 14. PROCESS BMD ONE IMAGE AT A TIME
# =====

for sample in tqdm(
    bmd_stream,
    total=BMD_LIMIT,
    desc="BMD-45 vehicles"
):

    if bmd_images >= BMD_LIMIT:
        break

    try:

        image = sample["image"]
        objects = sample["objects"]

        width, height = image.size

        bboxes = objects["bbox"]

        # Some HF datasets return class labels as integers.
        # We don't need the original class because everything
        # becomes generic vehicle class 0.

        if not bboxes:
            continue

        yolo_lines = []

        for bbox in bboxes:

            if len(bbox) != 4:
                continue

            x, y, w, h = [
                float(v)
                for v in bbox
            ]

            if w <= 0 or h <= 0:
                continue

            xmin = x
            ymin = y
            xmax = x + w
            ymax = y + h

            yolo_bbox = bbox_to_yolo(
                xmin,

```

```

        ymin,
        xmax,
        ymax,
        width,
        height
    )

    if yolo_bbox is None:
        continue

    xc, yc, bw, bh = yolo_bbox

    yolo_lines.append(
        f"0 {xc:.6f} {yc:.6f} {bw:.6f} {bh:.6f}"
    )

    if not yolo_lines:
        continue

    filename = f"bmd_{bmd_images:06d}.jpg"

    destination_image = (
        VEHICLE_ROOT
        / "images"
        / filename
    )

    destination_label = (
        VEHICLE_ROOT
        / "labels"
        / f"bmd_{bmd_images:06d}.txt"
    )

    # Save immediately.
    # Do NOT accumulate images in RAM.

    image.convert("RGB").save(
        destination_image,
        quality=95
    )

    destination_label.write_text(
        "\n".join(yolo_lines),
        encoding="utf-8"
    )

    bmd_images += 1
    bmd_boxes += len(yolo_lines)

except Exception as e:

    print(
        f"\nSkipped BMD sample: {e}"
    )

print("\nBMD processing complete.")
print(f"BMD images collected: {bmd_images}")
print(f"Vehicle boxes: {bmd_boxes}")

# =====
# 15. CREATE MERGED SOURCE POOL
# =====

print("\n=====")
print("CREATING MERGED SOURCE POOL")
print("=====")

merged_images = MERGED_ROOT / "images"
merged_labels = MERGED_ROOT / "labels"

merged_images.mkdir(
    parents=True,
    exist_ok=True
)

merged_labels.mkdir(
    parents=True,
    exist_ok=True
)

```

```

# -----
# Copy vehicle source images
# -----

for image_path in tqdm(
    list((VEHICLE_ROOT / "images").iterdir()),
    desc="Merging vehicles"
):

    if not image_path.is_file():
        continue

    destination = (
        merged_images
        / f"vehicle_{image_path.name}"
    )

    shutil.copy2(
        image_path,
        destination
    )

    source_label = (
        VEHICLE_ROOT
        / "labels"
        / f"{image_path.stem}.txt"
    )

    destination_label = (
        merged_labels
        / f"vehicle_{image_path.stem}.txt"
    )

    if source_label.exists():
        shutil.copy2(
            source_label,
            destination_label
        )

# -----
# Copy plate source images
# -----

for image_path in tqdm(
    list((PLATE_ROOT / "images").iterdir()),
    desc="Merging plates"
):

    if not image_path.is_file():
        continue

    destination = (
        merged_images
        / f"plate_{image_path.name}"
    )

    shutil.copy2(
        image_path,
        destination
    )

    source_label = (
        PLATE_ROOT
        / "labels"
        / f"{image_path.stem}.txt"
    )

    destination_label = (
        merged_labels
        / f"plate_{image_path.stem}.txt"
    )

    if source_label.exists():
        shutil.copy2(
            source_label,
            destination_label
        )

# =====

```

```

# 16. DATA.YAML
# =====

data_yaml = f"""# ParkFNB Vehicle + License Plate Dataset
# This is a SOURCE/SYNTHESIS POOL.
# Final train/val/test split will be created AFTER synthesis.

path: {MERGED_ROOT}

train: images
val: images

nc: 2

names:
  0: vehicle
  1: license_plate
"""

(MERGED_ROOT / "data.yaml").write_text(
    data_yaml,
    encoding="utf-8"
)

# =====
# 17. DATASET README
# =====

readme = f"""
# ParkFNB Dataset

## Purpose

Source dataset for ParkFNB ANPR synthetic data generation.

## Classes

0 = vehicle
1 = license_plate

## Vehicle source

BMD-45 – streamed from Hugging Face.

## License plate sources

1. Indian Vehicle Dataset
2. Kedarsai Indian License Plates

## Important

This is NOT the final training dataset.

The source pools are intentionally separated:

vehicle_base/
  vehicle images + class 0 vehicle boxes

plate_assets/
  plate images + class 1 plate boxes

merged_source/
  combined source pool

The final training dataset must be generated after synthetic
license plates are placed onto vehicle images.

Final train/val/test splitting should happen after synthesis.

## Collected

BMD vehicle images: {bmd_images}
BMD vehicle boxes: {bmd_boxes}

Indian/Kedarsai plate images:
{vehicle_count + kedarsai_count}

Plate boxes:
{vehicle_label_count + kedarsai_boxes}
"""

```

```

(DRIVE_ROOT / "README.md").write_text(
    readme,
    encoding="utf-8"
)

# =====
# 18. FINAL VALIDATION
# =====

print("\n=====")
print("FINAL DATASET VALIDATION")
print("=====")

def count_files(folder, extensions=None):

    folder = Path(folder)

    if not folder.exists():
        return 0

    count = 0

    for p in folder.iterdir():

        if not p.is_file():
            continue

        if extensions is None:
            count += 1

        elif p.suffix.lower() in extensions:
            count += 1

    return count

vehicle_images_final = count_files(
    VEHICLE_ROOT / "images",
    IMAGE_EXTENSIONS
)

vehicle_labels_final = count_files(
    VEHICLE_ROOT / "labels",
    {".txt"}
)

plate_images_final = count_files(
    PLATE_ROOT / "images",
    IMAGE_EXTENSIONS
)

plate_labels_final = count_files(
    PLATE_ROOT / "labels",
    {".txt"}
)

merged_images_final = count_files(
    MERGED_ROOT / "images",
    IMAGE_EXTENSIONS
)

merged_labels_final = count_files(
    MERGED_ROOT / "labels",
    {".txt"}
)

print("\n-----")
print("VEHICLE SOURCE")
print("-----")
print("Images :", vehicle_images_final)
print("Labels :", vehicle_labels_final)

print("\n-----")
print("PLATE SOURCE")
print("-----")
print("Images :", plate_images_final)
print("Labels :", plate_labels_final)

print("\n-----")

```

```
print("MERGED SOURCE")
print("-----")
print("Images :", merged_images_final)
print("Labels :", merged_labels_final)

print("\n-----")
print("DRIVE LOCATION")
print("-----")
print(DRIVE_ROOT)

print("\n=====")
print("DONE")
print("=====")

print("""
NO TRAINING WAS RUN.
NO GPU WAS REQUIRED.
NO FINAL TRAIN/VAL/TEST SPLIT WAS CREATED.

Next stage:
VEHICLE BASE + PLATE ASSETS
    ↓
SYNTHETIC PLATE COMPOSITING
    ↓
IMAGE + class 0 vehicle boxes
      + class 1 plate boxes
    ↓
FINAL DATASET
    ↓
TRAIN / VAL / TEST
    ↓
YOLO TRAINING
""")
```

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=

=====
CLEANING OLD PARKFNB DATASET
=====
Deleting:
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset

OLD DATASET DELETED.
Fresh workspace created.

=====
DOWNLOADING INDIAN VEHICLE DATASET
=====

Downloading:
https://www.kaggle.com/api/v1/datasets/download/saisirishan/indian-vehicle-dataset
Saved: /content/parkfnb_work/kaggle/indian_vehicle_dataset.zip

=====
DOWNLOADING KEDARSAI PLATE DATASET
=====

Downloading:
https://www.kaggle.com/api/v1/datasets/download/kedarsai/indian-license-plates-with-labels
Saved: /content/parkfnb_work/kaggle/kedarsai_plates.zip

Extracting Indian Vehicle Dataset...
Extracting Kedarsai Dataset...

Kaggle datasets extracted.

=====
PROCESSING INDIAN VEHICLE DATASET
=====
XML files found: 1697
Indian Vehicle XML: 100%                                1697/1697 [00:45<00:00, 36.76it/s]

Indian Vehicle Dataset processing complete.
Images with plate labels: 1694
License plate boxes:      1694

=====
PROCESSING KEDARSAI PLATE DATASET
=====
Images found: 2083
TXT labels found: 2021
Kedarsai plates: 100%                                2083/2083 [00:48<00:00, 46.94it/s]

Kedarsai processing complete.
Images with plate labels: 2021
License plate boxes:      2168

=====
STREAMING BMD-45

```

```

import torch

print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
print("GPU count:", torch.cuda.device_count())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
else:
    print("❌ GPU is NOT enabled")

```

```

# =====
# PARKFNB - YOLO11 TRAINING ON GOOGLE COLAB
# DATA FROM GOOGLE DRIVE
# =====

# =====
# 1. INSTALL ULTRALYTICS
# =====

!pip -q install -U ultralytics

# =====
# 2. MOUNT GOOGLE DRIVE
# =====

from google.colab import drive
drive.mount('/content/drive')

```

```

# =====
# 3. IMPORTS
# =====

from pathlib import Path
import yaml
import os
import torch

from ultralytics import YOLO

# =====
# 4. PATHS
# =====

DATASET_ROOT = Path(
    "/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset"
)

DATA_YAML = DATASET_ROOT / "data.yaml"

RUNS_DIR = DATASET_ROOT / "training_runs"

RUNS_DIR.mkdir(
    parents=True,
    exist_ok=True
)

# =====
# 5. CHECK GPU
# =====

print("=====")
print("GPU CHECK")
print("=====")

print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():

    print("GPU:", torch.cuda.get_device_name(0))

    DEVICE = 0

else:

    print("WARNING: GPU NOT AVAILABLE")
    print("Go to:")
    print("Runtime → Change runtime type → T4 GPU")

    DEVICE = "cpu"

# =====
# 6. CHECK DATASET
# =====

print("\n=====")
print("DATASET CHECK")
print("=====")

print("Dataset:", DATASET_ROOT)
print("YAML:", DATA_YAML)

if not DATASET_ROOT.exists():
    raise FileNotFoundError(
        f"Dataset folder not found:\n{DATASET_ROOT}"
    )

if not DATA_YAML.exists():
    raise FileNotFoundError(
        f"data.yaml not found:\n{DATA_YAML}"
    )

# =====
# 7. LOAD YAML
# =====

```

```

with open(DATA_YAML, "r", encoding="utf-8") as f:
    data_config = yaml.safe_load(f)

print("\nDataset configuration:")
print(data_config)

# =====
# 8. EXPECTED CLASSES
# =====

expected_names = {
    0: "vehicle",
    1: "license_plate"
}

names = data_config.get("names")

print("\nClasses:")
print(names)

# Convert list → dictionary if required
if isinstance(names, list):
    actual_names = {
        i: name
        for i, name in enumerate(names)
    }
else:
    actual_names = names

if actual_names != expected_names:
    print("\nWARNING:")
    print("Expected:", expected_names)
    print("Found:   ", actual_names)
else:
    print("\nClass configuration OK.")

# =====
# 9. DATASET STRUCTURE CHECK
# =====

print("\n=====")
print("DIRECTORY CHECK")
print("=====")

required_dirs = [
    DATASET_ROOT / "images" / "train",
    DATASET_ROOT / "images" / "val",
    DATASET_ROOT / "images" / "test",

    DATASET_ROOT / "labels" / "train",
    DATASET_ROOT / "labels" / "val",
    DATASET_ROOT / "labels" / "test",
]

for folder in required_dirs:

    if folder.exists():

        files = list(folder.iterdir())

        print(
            f"OK   {folder} "
            f"({len(files)} files)"
        )

    else:

        print(
            f"MISSING {folder}"
        )

# =====
# 10. COUNT IMAGES
# =====

IMAGE_EXTENSIONS = {
    ".jpg",
    ".jpeg",

```

```

        ".png",
        ".webp",
        ".bmp"
    }

def count_images(folder):

    folder = Path(folder)

    if not folder.exists():
        return 0

    return sum(
        1
        for p in folder.iterdir()
        if p.is_file()
        and p.suffix.lower() in IMAGE_EXTENSIONS
    )

train_images = count_images(
    DATASET_ROOT / "images" / "train"
)

val_images = count_images(
    DATASET_ROOT / "images" / "val"
)

test_images = count_images(
    DATASET_ROOT / "images" / "test"
)

print("\n=====")
print("IMAGE COUNTS")
print("=====")

print("Train:", train_images)
print("Val:  ", val_images)
print("Test: ", test_images)

if train_images == 0:
    raise RuntimeError(
        "Training images not found."
    )

if val_images == 0:
    raise RuntimeError(
        "Validation images not found."
    )

# =====
# 11. LOAD YOLO11n
# =====

print("\n=====")
print("LOADING YOLO11n")
print("=====")

model = YOLO("yolo11n.pt")

print("YOLO11n loaded.")

# =====
# 12. TRAIN
# =====

print("\n=====")
print("STARTING PARKFNB TRAINING")
print("=====")

results = model.train(

    # Dataset
    data=str(DATA_YAML),

    # Model
    model="yolo11n.pt",

    # Training

```

```

epochs=100,
imgsz=640,

# Hardware
device=DEVICE,

# Batch
batch=-1,

# Workers
workers=2,

# Output
project=str(RUNS_DIR),
name="yolo11n_vehicle_plate",

# Save
save=True,
save_period=10,

# Validation
val=True,

# Cache
cache=False,

# Reproducibility
seed=42,

# Early stopping
patience=20,

# Logging
verbose=True
)

# =====
# 13. BEST MODEL LOCATION
# =====

BEST_MODEL = (
    RUNS_DIR
    / "yolo11n_vehicle_plate"
    / "weights"
    / "best.pt"
)

LAST_MODEL = (
    RUNS_DIR
    / "yolo11n_vehicle_plate"
    / "weights"
    / "last.pt"
)

print("\n=====")
print("TRAINING COMPLETE")
print("=====")

print("Best model:")
print(BEST_MODEL)

print("\nLast model:")
print(LAST_MODEL)

print("\nBest exists:", BEST_MODEL.exists())
print("Last exists:", LAST_MODEL.exists())

# =====
# 14. FINAL MESSAGE
# =====

print("""
=====
PARKFNB YOLO TRAINING FINISHED
=====

Classes:

0 → vehicle
1 → license_plate

```

```

Model:

YOLO11n

Outputs are stored permanently in Google Drive:

ParkFNB/
├── ParkFNB_Dataset/
│   └── training_runs/
│       └── yolo11n_vehicle_plate/
│           ├── weights/
│           │   ├── best.pt
│           │   └── last.pt
│           ├── results.png
│           └── results.csv
│           ...
NEXT:
best.pt
↓
CCTV video testing
↓
vehicle detection
↓
license plate detection
↓
OCR
↓
parking-space occupancy
↓
ParkFNB backend
"""

```

```

Mounted at /content/drive
Creating new Ultralytics Settings v0.0.8 file ✓
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com
=====
GPU CHECK
=====
CUDA available: True
GPU: Tesla T4

=====
DATASET CHECK
=====
Dataset: /content/drive/MyDrive/ParkFNB/ParkFNB_Dataset
YAML: /content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/data.yaml
-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_2952/1828169302.py in <cell line: 0>()
     91
     92 if not DATA_YAML.exists():
--> 93     raise FileNotFoundError(
     94         f"data.yaml not found:\n{DATA_YAML}"
     95     )

FileNotFoundError: data.yaml not found:
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/data.yaml

```

```

from pathlib import Path

BASE = Path("/content/drive/MyDrive/ParkFNB")

print("ParkFNB folders/files:\n")

for p in BASE.rglob("*"):
    if p.is_file():
        print(p)

```

```

Streaming output truncated to the last 5000 lines.
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001452.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001453.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001454.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001455.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001456.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001457.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001458.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001459.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001460.jpg

```

```

/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001461.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001462.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001463.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001464.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001465.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001466.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001467.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001468.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001469.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001470.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001471.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001472.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001473.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001474.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001475.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001476.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001477.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001478.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001479.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001480.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001481.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001482.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001483.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001484.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001485.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001486.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001487.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001488.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001489.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001490.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001491.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001492.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001493.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001494.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001495.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001496.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001497.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001498.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001499.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001500.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001501.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001502.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001503.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001504.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001505.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001506.jpg
/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/vehicle_base/images/bmd_001507.jpg

```

```

# =====
# PARKFNB – SPLIT MERGED POOL INTO TRAIN/VAL/TEST
# (run this BEFORE the training cell)
# =====
from pathlib import Path
import shutil
import random

random.seed(42)

DRIVE_ROOT = Path("/content/drive/MyDrive/ParkFNB")
DATASET_ROOT = DRIVE_ROOT / "ParkFNB_Dataset"
MERGED_ROOT = DATASET_ROOT / "merged_source"

for split in ["train", "val", "test"]:
    (DATASET_ROOT / "images" / split).mkdir(parents=True, exist_ok=True)
    (DATASET_ROOT / "labels" / split).mkdir(parents=True, exist_ok=True)

pairs = []
for img in sorted((MERGED_ROOT / "images").iterdir()):
    if not img.is_file():
        continue
    label = MERGED_ROOT / "labels" / f"{img.stem}.txt"
    if label.exists():
        pairs.append((img, label))

print("Total labeled pairs in merged pool:", len(pairs))

random.shuffle(pairs)
n = len(pairs)
train_end = int(n * 0.8)
val_end = train_end + int(n * 0.1)

splits = {
    "train": pairs[:train_end],
    "val": pairs[train_end:val_end],
    "test": pairs[val_end:],
}

```

```

for split, items in splits.items():
    print(f"Copying {split}: {len(items)}")
    for img, label in items:
        shutil.copy2(img, DATASET_ROOT / "images" / split / img.name)
        shutil.copy2(label, DATASET_ROOT / "labels" / split / label.name)

# data.yaml AT THE ROOT - this is what the training cell expects
yaml_text = f"""path: {DATASET_ROOT}
train: images/train
val: images/val
test: images/test

nc: 2
names:
  0: vehicle
  1: license_plate
"""
(DATASET_ROOT / "data.yaml").write_text(yaml_text, encoding="utf-8")

print("\nSplit complete.")
for split in ["train", "val", "test"]:
    n_img = len(list((DATASET_ROOT / "images" / split).iterdir()))
    print(f"{split}: {n_img} images")
print("\ndata.yaml written to:", DATASET_ROOT / "data.yaml")

```

Total labeled pairs in merged pool: 3354
Copying train: 2683

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipykernel_2952/1764979179.py in <cell line: 0>()
    42     for img, label in items:
    43         shutil.copy2(img, DATASET_ROOT / "images" / split / img.name)
--> 44         shutil.copy2(label, DATASET_ROOT / "labels" / split / label.name)
    45
    46 # data.yaml AT THE ROOT - this is what the training cell expects

```

↕ 2 frames

```

/usr/lib/python3.13/shutil.py in _fastcopy_sendfile(fsrc, fdst)
    148     while True:
    149         try:
--> 150             sent = os.sendfile(outfd, infd, offset, blocksize)
    151         except OSError as err:
    152             # ...in order to have a more informative exception.

```

KeyboardInterrupt:

```

# =====
# PARKFNB - FAST LOCAL SPLIT (no repeated Drive I/O)
# Reads from Drive merged_source ONCE, builds split
# on local disk, then zips ONE file back to Drive.
# =====
from pathlib import Path
import shutil
import random
from tqdm.auto import tqdm

random.seed(42)

DRIVE_ROOT = Path("/content/drive/MyDrive/ParkFNB")
MERGED_ROOT = DRIVE_ROOT / "ParkFNB_Dataset" / "merged_source"

LOCAL_ROOT = Path("/content/parkfnn_split")
if LOCAL_ROOT.exists():
    shutil.rmtree(LOCAL_ROOT)
for split in ["train", "val", "test"]:
    (LOCAL_ROOT / "images" / split).mkdir(parents=True, exist_ok=True)
    (LOCAL_ROOT / "labels" / split).mkdir(parents=True, exist_ok=True)

# Read the file list once (fast, metadata-only)
pairs = []
for img in sorted((MERGED_ROOT / "images").iterdir()):
    if not img.is_file():
        continue
    label = MERGED_ROOT / "labels" / f"{img.stem}.txt"
    if label.exists():
        pairs.append((img, label))

print("Total labeled pairs found:", len(pairs))

```

```

random.shuffle(pairs)
n = len(pairs)
train_end = int(n * 0.8)
val_end = train_end + int(n * 0.1)
splits = {
    "train": pairs[:train_end],
    "val": pairs[train_end:val_end],
    "test": pairs[val_end:],
}

for split, items in splits.items():
    for img, label in tqdm(items, desc=f"Copying {split}"):
        shutil.copy2(img, LOCAL_ROOT / "images" / split / img.name)
        shutil.copy2(label, LOCAL_ROOT / "labels" / split / label.name)

yaml_text = f"""path: /content/drive/MyDrive/ParkFNB/ParkFNB_Dataset
train: images/train
val: images/val
test: images/test

nc: 2
names:
  0: vehicle
  1: license_plate
"""
(LOCAL_ROOT / "data.yaml").write_text(yaml_text, encoding="utf-8")

# Single zip -> single Drive write, instead of thousands of small writes
shutil.make_archive("/content/parkfnn_split_final", "zip", LOCAL_ROOT)
shutil.copy2("/content/parkfnn_split_final.zip", DRIVE_ROOT / "parkfnn_split_final.zip")

print("\nDone. Zip saved to Drive:", DRIVE_ROOT / "parkfnn_split_final.zip")
print("Extract it into ParkFNB_Dataset/ before training, or point training directly at:")
print(LOCAL_ROOT)

```

Total labeled pairs found: 3354

Copying train: 100% 2683/2683 [16:29<00:00, 1.28it/s]

Copying val: 100% 335/335 [04:55<00:00, 1.23it/s]

Copying test: 100% 336/336 [04:47<00:00, 1.15it/s]

Done. Zip saved to Drive: /content/drive/MyDrive/ParkFNB/parkfnn_split_final.zip
 Extract it into ParkFNB_Dataset/ before training, or point training directly at:
 /content/parkfnn_split

```

from pathlib import Path

DRIVE_ROOT = Path("/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset")

for name in ["vehicle_base", "plate_assets", "merged_source"]:
    folder = DRIVE_ROOT / name
    n_img = len(list((folder / "images").glob("*")))
    n_lbl = len(list((folder / "labels").glob("*.txt")))
    print(f"{name:15} | images: {n_img:5} | labels: {n_lbl:5}")

```

```

vehicle_base | images: 3817 | labels: 3817
plate_assets | images: 3715 | labels: 3715
merged_source | images: 3355 | labels: 3354

```

```

# =====
# PARKFNB - DIRECT SPLIT FROM SOURCES (skip merged_source)
# Reads vehicle_base + plate_assets (both confirmed complete),
# builds train/val/test locally with PARALLEL reads (faster on Drive),
# then uploads ONE final zip to Drive.
# =====
from pathlib import Path
import shutil
import random
from concurrent.futures import ThreadPoolExecutor, as_completed
from tqdm.auto import tqdm

random.seed(42)

DRIVE_ROOT = Path("/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset")
VEHICLE_ROOT = DRIVE_ROOT / "vehicle_base"
PLATE_ROOT = DRIVE_ROOT / "plate_assets"

LOCAL_ROOT = Path("/content/parkfnn_split")
if LOCAL_ROOT.exists():
    shutil.rmtree(LOCAL_ROOT)

```

```

for split in ["train", "val", "test"]:
    (LOCAL_ROOT / "images" / split).mkdir(parents=True, exist_ok=True)
    (LOCAL_ROOT / "labels" / split).mkdir(parents=True, exist_ok=True)

# -----
# Build the full pair list directly from BOTH sources
# -----
pairs = []

for img in sorted((VEHICLE_ROOT / "images").iterdir()):
    if not img.is_file():
        continue
    label = VEHICLE_ROOT / "labels" / f"{img.stem}.txt"
    if label.exists():
        pairs.append((img, label, "vehicle"))

for img in sorted((PLATE_ROOT / "images").iterdir()):
    if not img.is_file():
        continue
    label = PLATE_ROOT / "labels" / f"{img.stem}.txt"
    if label.exists():
        pairs.append((img, label, "plate"))

print("Total labeled pairs found:", len(pairs))

random.shuffle(pairs)
n = len(pairs)
train_end = int(n * 0.8)
val_end = train_end + int(n * 0.1)
splits = {
    "train": pairs[:train_end],
    "val": pairs[train_end:val_end],
    "test": pairs[val_end:],
}

def copy_pair(item, split):
    img, label, prefix = item
    shutil.copy2(img, LOCAL_ROOT / "images" / split / f"{prefix}_{img.name}")
    shutil.copy2(label, LOCAL_ROOT / "labels" / split / f"{prefix}_{img.stem}.txt")

# -----
# Parallel copy – Drive I/O is latency-bound, so many
# concurrent requests are much faster than sequential
# -----
for split, items in splits.items():
    with ThreadPoolExecutor(max_workers=16) as ex:
        futures = [ex.submit(copy_pair, item, split) for item in items]
        for _ in tqdm(as_completed(futures), total=len(futures), desc=f"Copying {split}"):
            pass

yaml_text = f"""path: /content/drive/MyDrive/ParkFNB/ParkFNB_Dataset
train: images/train
val: images/val
test: images/test

nc: 2
names:
  0: vehicle
  1: license_plate
"""
(LOCAL_ROOT / "data.yaml").write_text(yaml_text, encoding="utf-8")

shutil.make_archive("/content/parkfnn_split_final", "zip", LOCAL_ROOT)
shutil.copy2("/content/parkfnn_split_final.zip", DRIVE_ROOT / "parkfnn_split_final.zip")

print("\nDone. Total pairs:", n)
for split, items in splits.items():
    print(f"{split}: {len(items)}")
print("Zip saved to Drive:", DRIVE_ROOT / "parkfnn_split_final.zip")
print("Local (fast) copy for training:", LOCAL_ROOT)

```

```

Total labeled pairs found: 7532
Copying train: 100%                                6025/6025 [09:17<00:00, 11.60it/s]
Copying val: 100%                                  753/753 [01:08<00:00, 8.68it/s]
Copying test: 100%                                754/754 [01:09<00:00, 5.88it/s]

Done. Total pairs: 7532
train: 6025
val: 753
test: 754
Zip saved to Drive: /content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/parkfnn_split_final.zip
Local (fast) copy for training: /content/parkfnn_split

```

```

DATASET_ROOT = Path("/content/parkfnn_split") # local, not Drive
DATA_YAML = DATASET_ROOT / "data.yaml"

```

```

# =====
# PARKFNB - FINAL TRAINING (plate-focused tuning, fixed)
# YOLO11n | Vehicle + License Plate
# =====
import sys, subprocess
import torch
from pathlib import Path

try:
    from ultralytics import YOLO
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "ultralytics"])
    from ultralytics import YOLO

DATA_YAML = Path("/content/parkfnn_split/data.yaml")
assert DATA_YAML.exists(), "data.yaml missing"
yaml_text = DATA_YAML.read_text()
print(yaml_text)
assert "nc: 2" in yaml_text, "nc is not 2 - fix dataset before training"

print("CUDA available:", torch.cuda.is_available())
DEVICE = 0 if torch.cuda.is_available() else "cpu"
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

model = YOLO("yolo11n.pt")

results = model.train(
    data=str(DATA_YAML),
    epochs=100,
    imgsz=960,
    batch=-1,
    device=DEVICE,

    optimizer="AdamW",
    lr=0.001,
    lrf=0.01,
    weight_decay=0.0005,
    warmup_epochs=3,
    warmup_momentum=0.8,
    cos_lr=True,

    patience=25,

    multi_scale=False, # FIXED - was forcing autobatch down to batch=2
    scale=0.5,

    hsv_h=0.015,
    hsv_s=0.50,
    hsv_v=0.40,
    degrees=3,
    translate=0.08,
    shear=0.0,
    perspective=0.0002,
    flipplr=0.5,
    flipud=0.0,
    mosaic=1.0,
    mixup=0.05,
    close_mosaic=15,

    amp=True,
    workers=2,
    cache=False,

```

```

seed=42,
deterministic=True,

project="/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/training_runs",
name="yolo11n_vehicle_plate_v3",
exist_ok=True,
save=True,
save_period=5,
val=True,
)

print("\n=====")
print("TRAINING COMPLETE")
print("=====")

metrics = model.val(data=str(DATA_YAML), imgsiz=960, device=DEVICE)

print("Overall mAP50      :", metrics.box.map50)
print("Overall mAP50-95   :", metrics.box.map)
print("Overall Precision   :", metrics.box.mp)
print("Overall Recall      :", metrics.box.mr)

print("\nPer-class mAP50-95:")
class_names = model.names
for i, ap in enumerate(metrics.box.maps):
    print(f"  {class_names[i]:15}: {ap:.4f}")

print("\nBest model:")
print("/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/training_runs/yolo11n_vehicle_plate_v3/weights/best.pt")

```

```

path: /content/parkfnn_split
train: images/train
val: images/val
test: images/test

```

```

nc: 2
names:
  0: vehicle
  1: license_plate

```

CUDA available: True

GPU: Tesla T4

New <https://pypi.org/project/ultralytics/8.4.149> available 🤗 Update with 'pip install -U ultralytics'

Ultralytics 8.4.148 🐍 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugument, batch=-1, bgr=0.0, box=7

Overriding model.yaml nc=80 with nc=2

	from	n	params	module	arguments
0	-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
1	-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
2	-1	1	6640	ultralytics.nn.modules.block.C3k2	[32, 64, 1, False, 0.25]
3	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
4	-1	1	26080	ultralytics.nn.modules.block.C3k2	[64, 128, 1, False, 0.25]
5	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
6	-1	1	87040	ultralytics.nn.modules.block.C3k2	[128, 128, 1, True]
7	-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
8	-1	1	346112	ultralytics.nn.modules.block.C3k2	[256, 256, 1, True]
9	-1	1	164608	ultralytics.nn.modules.block.SPPF	[256, 256, 5]
10	-1	1	249728	ultralytics.nn.modules.block.C2PSA	[256, 256, 1]
11	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
12	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
13	-1	1	111296	ultralytics.nn.modules.block.C3k2	[384, 128, 1, False]
14	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
15	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
16	-1	1	32096	ultralytics.nn.modules.block.C3k2	[256, 64, 1, False]
17	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
18	[-1, 13]	1	0	ultralytics.nn.modules.conv.Concat	[1]
19	-1	1	86720	ultralytics.nn.modules.block.C3k2	[192, 128, 1, False]
20	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
21	[-1, 10]	1	0	ultralytics.nn.modules.conv.Concat	[1]
22	-1	1	378880	ultralytics.nn.modules.block.C3k2	[384, 256, 1, True]
23	[16, 19, 22]	1	431062	ultralytics.nn.modules.head.Detect	[2, 16, None, [64, 128, 256]]

YOLO11n summary: 181 layers, 2,590,230 parameters, 2,590,214 gradients, 6.5 GFLOPs

Transferred 448/499 items from pretrained weights

Freezing layer 'model.23.dfl.conv.weight'

AMP: running Automatic Mixed Precision (AMP) checks...

AMP: checks passed ✅

train: Fast image access ✅ (ping: 0.0±0.0 ms, read: 330.3±572.7 MB/s, size: 149.2 KB)

train: Scanning /content/parkfnn_split/labels/train... 6025 images, 0 backgrounds, 0 corrupt: 100% ————— 6025/6025

train: New cache created: /content/parkfnn_split/labels/train.cache

augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_av

AutoBatch: Computing optimal batch size for imgsiz=960 at 60.0% GPU memory utilization.

AutoBatch: CUDA:0 (Tesla T4) 14.56G total, 2.02G reserved, 0.08G allocated, 12.46G free

Params	GFLOPs	GPU_mem (GB)	forward (ms)	backward (ms)	input	output
2590230	14.49	0.801	34.19	nan	(1, 3, 960, 960)	list

2590230

28.98

1.810

38.13

nan

(2. 3. 960. 960)

list

```
# =====
# FIX data.yaml – point to the ACTUAL local dataset
# =====
from pathlib import Path

LOCAL_ROOT = Path("/content/parkfmb_split")

yaml_text = f"""path: {LOCAL_ROOT}
train: images/train
val: images/val
test: images/test

nc: 2
names:
  0: vehicle
  1: license_plate
"""

(LOCAL_ROOT / "data.yaml").write_text(yaml_text, encoding="utf-8")
print((LOCAL_ROOT / "data.yaml").read_text())

# Quick sanity check before training
for split in ["train", "val", "test"]:
    n = len(list((LOCAL_ROOT / "images" / split).glob("*")))
    print(f"{split}: {n} images")
```

```
path: /content/parkfmb_split
train: images/train
val: images/val
test: images/test

nc: 2
names:
  0: vehicle
  1: license_plate

train: 6025 images
val: 753 images
test: 754 images
```

```
# =====
# PARKFNB – RESUME TRAINING (data.yaml auto-fixed every time)
# =====
from google.colab import drive
drive.mount("/content/drive")

import sys, subprocess, zipfile
from pathlib import Path

try:
    from ultralytics import YOLO
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "ultralytics"])
    from ultralytics import YOLO

# -----
# 1. Dataset wapad local disk par laao
# -----
ZIP_PATH = "/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/parkfmb_split_final.zip"
LOCAL_ROOT = Path("/content/parkfmb_split")

if not LOCAL_ROOT.exists():
    print("Extracting dataset...")
    with zipfile.ZipFile(ZIP_PATH) as z:
        z.extractall(LOCAL_ROOT)
    print("Done.")
else:
    print("Dataset already present, skipping extract.")

# -----
# 2. data.yaml ko FORCE overwrite karo – zip ke andar wali purani,
# galat (Drive-pointing) file par bharosa mat karo
# -----
yaml_text = f"""path: {LOCAL_ROOT}
train: images/train
val: images/val
test: images/test

nc: 2
names:
  0: vehicle
  1: license_plate
```

```

"""
(LOCAL_ROOT / "data.yaml").write_text(yaml_text, encoding="utf-8")
print("\ndata.yaml rewritten:")
print((LOCAL_ROOT / "data.yaml").read_text())

# Sanity check
for split in ["train", "val", "test"]:
    n = len(list((LOCAL_ROOT / "images" / split).glob("*.jpg")))
    print(f"{split}: {n} images")

# -----
# 3. Last checkpoint se resume karo
# -----
LAST_CKPT = "/content/drive/MyDrive/ParkFNB/ParkFNB_Dataset/training_runs/yolo11n_vehicle_plate_v3/weights/last.pt"
assert Path(LAST_CKPT).exists(), f"Checkpoint not found: {LAST_CKPT}"

model = YOLO(LAST_CKPT)
results = model.train(resume=True)

```

9	1	1	256	ultralalytics.nn.modules.block.C2PSA	[256, 256, 1]
10	-1	1	249728	ultralalytics.nn.modules.block.C2PSA	[256, 256, 1]
11	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
12	[-1, 6]	1	0	ultralalytics.nn.modules.conv.Concat	[1]
13	-1	1	111296	ultralalytics.nn.modules.block.C3k2	[384, 128, 1, False]
14	1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']